Cloud Computing Suite: A Multi-Model Cloud Deployment with CI/CD and Infrastructure as Code

Arjit Menon
School of Computer Engineering
Manipal Institute of Technology
Bengaluru, India

Dhruvishi Amin
School of Computer Engineering
Manipal Institute of Technology
Bengaluru, India

Arisha Jaiswal
School of Computer Engineering
Manipal Institute of Technology
Bengaluru, India

Manya Vishwanath
School of Computer Engineering
Manipal Institute of Technology
Bengaluru, India

Swasti Gupta
School of Computer Engineering
Manipal Institute of Technology
Bengaluru, India

***Abstract*—Most student cloud projects deploy one CRUD application to one server, so they exercise a single deployment model. This paper describes Cloud Computing Suite, a small system of three independently deployed services that together use three compute models. The first, utility-hub, is an always-on web service that shortens URLs and serves a read API from a managed Redis store. The second, stats-cron, is a scheduled job that runs every six hours on GitHub Actions, writes a statistics digest to the same store and exits. The third, portfolio, is a static page served from a CDN that monitors the other two from the browser. Each service has its own Dockerfile, is tested in a GitHub Actions pipeline against a real Redis instance, and is described in Terraform using the Render provider. The whole system runs on free tiers. All 12 automated tests pass and the live deployment reports a healthy Redis connection. We also discuss how the free-tier constraint shaped the design and list the gaps a production version would have to close.**

Keywords—cloud computing, DevOps, continuous integration, infrastructure as code, Terraform, Docker, Redis, scheduled jobs, platform as a service

I. Introduction

Cloud applications are rarely a single process on a single machine. A typical system mixes long-running services, short jobs that run on a timer and static content, and it picks a storage engine for each workload rather than one for everything [1], [2]. It also depends on automation: tests run on every change, images are built the same way each time, and infrastructure is written down as code instead of being clicked together in a dashboard.

A course project that deploys one CRUD application to one server shows only a small part of this. It does not force any decision about which compute model fits which job, which store fits which access pattern, or how a change gets from a commit to production. We wanted a project small enough for a team of students to build and run, but broad enough that these decisions had to be made and could be explained.

Cloud Computing Suite is the result. It has three services. utility-hub is an Express application on Node.js 20 that shortens URLs, counts clicks and serves a data digest. stats-cron is a short-lived job that reads repository statistics from the GitHub REST API and writes them to Redis. portfolio is a static page that checks the health of the other services from the visitor's browser. The services share one managed Redis store and are deployed separately.

The main contributions of this work are: a working three-service deployment that covers always-on, scheduled and static compute at zero cost; a CI pipeline that tests both back-end services against a real Redis instance before building their container images; a Terraform description of the hosted resources; and a list of the limitations that the free-tier constraint introduced.

The rest of the paper is organised as follows. Section II reviews related work. Section III states the problem and objectives. Sections IV to VI describe the architecture, the implementation and the DevOps pipeline. Section VII looks at the free tiers the system depends on. Section VIII reports test and deployment results, Section IX covers security, reliability and limitations, and Section X concludes. The source code is public [3].

II. Related Work

The NIST definition characterises cloud computing by on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service, delivered as infrastructure, platform or software as a service [1]. Our system sits mostly at the platform layer: we hand the platform source code or a container and it takes care of machines, networking and TLS.

Serverless computing moves further in that direction. Jonas et al. describe it as a model in which the provider runs short, event-triggered functions and bills only for the time they run [4]. stats-cron follows the same pattern, triggered by a timer instead of a request, although it runs on a CI runner rather than a function platform.

Lewis and Fowler describe microservices as small services that are deployed independently and own their data [2]. Dragoni et al. review the approach and point out that the benefits come with costs in operations and testing [5]. Our services are far smaller than the systems these papers have in mind, but the same trade-off appears: three deployments are more work to operate than one. The Twelve-Factor methodology adds practical rules that we follow, most importantly that configuration such as connection strings belongs in the environment and not in the code [6].

Merkel presents Docker as a way to package an application with its dependencies so that it runs identically on a laptop and on a server [7]. Continuous integration, as described by Fowler, requires every change to be built and tested automatically [8], and Humble and Farley extend this into continuous delivery, where the main branch is always releasable [9]. Shahin et al. survey these practices and note that slow or unreliable test suites are one of the main obstacles

to adopting them [10], which influenced our choice to keep the test suites small and fast.

Infrastructure as Code treats infrastructure definitions as source files that are versioned and reviewed like application code [11]. Rahman et al. map the research in this area and find that most of it concerns configuration tools such as Puppet and Ansible, with comparatively little on defects and quality in IaC scripts [12]. Our Terraform configuration is small, but its one unverified attribute (Section VI) is a small example of the kind of defect they describe.

III. Problem Statement and Objectives

The project had to demonstrate more than one deployment pattern, run with as little manual work as possible, and cost nothing. A single application on a single server would satisfy the last point but not the first, and would hide the decisions we were most interested in.

We set four objectives:

1) Use at least four managed cloud services or tools.

2) Demonstrate three compute models: an always-on web service, scheduled compute and static hosting.

3) Automate testing and image builds through CI, and describe the hosted infrastructure as code.

4) Run entirely on free tiers, with no paid resources and no stored payment details.

IV. System Architecture

Fig. 1 shows the overall design. The browser loads the static portfolio page, which calls utility-hub. utility-hub reads and writes the shared Redis store. stats-cron runs on a timer, calls the GitHub REST API and writes a digest into the same store, from which utility-hub serves it. stats-cron never receives traffic; it only writes.

Table I lists the cloud services and tools the system uses and the role each plays. The hosting platform is Render [13]; scheduled compute and CI both run on GitHub Actions [14].

TABLE I. Cloud Services and Tools Used

#	Service / tool	Category	Role in the system
1	Render Web Service	PaaS compute	Hosts utility-hub as an always-on Node.js service
2	Render Key Value (Redis)	Managed data store	Short URLs, click counters and the latest digest
3	Render Static Site	Static hosting and CDN	Serves the portfolio page
4	GitHub Actions (CI)	CI pipeline	Tests on a real Redis, then builds Docker images
5	GitHub Actions (schedule)	Scheduled compute	Runs stats-cron every 6 hours
6	Terraform with the Render provider	Infrastructure as Code	Declares the Redis store, web service and static site
7	Docker and Docker Compose	Containerisation	Reproducible images and a local stack
8	prom-client and pino	Observability	Prometheus metrics and JSON request logs

A. Choice of Data Store

The URL shortener only ever looks values up by a known key. There are no joins, no ad hoc queries and no relations between records, so a relational schema would add work without adding anything. Redis fits the access pattern directly: a GET for a lookup, a SET to create a record, and an INCR to count a click atomically [15]. Using the same store for the digest kept the number of hosted resources down to one.

B. Choice of Scheduler

Render offers a cron job resource, but it is not available on the free plan. A GitHub Actions workflow with a schedule trigger gives the same behaviour (start on a timer, do one task, exit) without a paid plan. The design does not depend on the platform: the same job could run as an AWS Lambda function triggered by EventBridge, or under Google Cloud Scheduler, with only the trigger configuration changing.

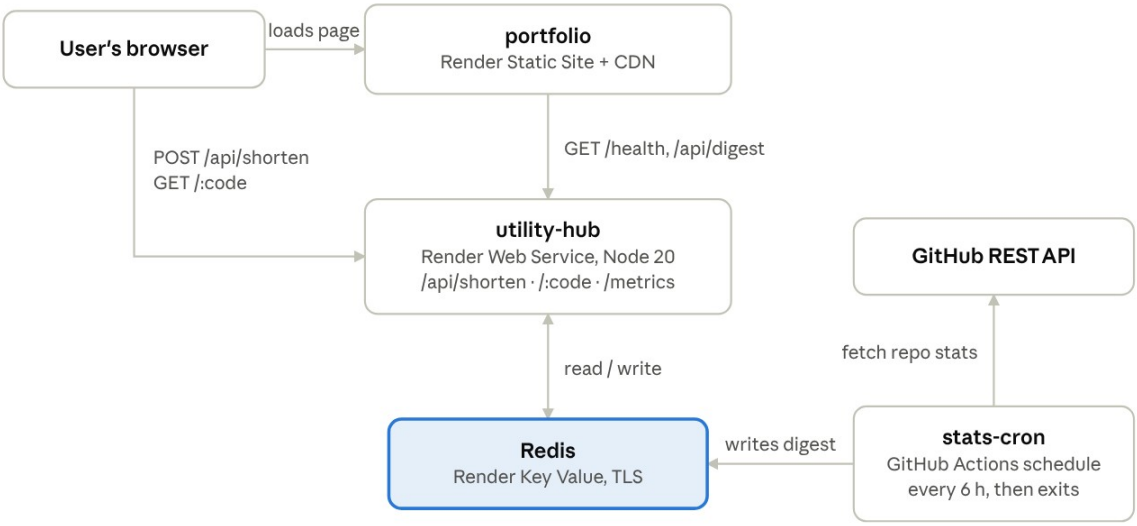


Fig. 1. System architecture. Three services share one managed Redis store.

C. Service Boundaries

Each service has its own directory, package.json, Dockerfile, CI job and deployment. A change to the portfolio page does not rebuild utility-hub, and a failing stats-cron run does not affect the web service, which keeps serving the last digest it has. The services are coupled only through two things:

the HTTP API that portfolio calls, and the Redis key that stats-cron writes and utility-hub reads.

V. Implementation

All application code is JavaScript on Node.js 20. Each service is a separate npm package inside one repository.

A. *utility-hub*

utility-hub is an Express 4 application. The file app.js builds the application around a Redis client that is passed in rather than created inside, which lets the tests supply their own client and keeps server.js down to a few lines that create the client, build the app and listen on a port. A shared module, redis.js, creates an ioredis client from the REDIS_URL environment variable and enables TLS when REDIS_TLS is set to true, which Render's managed Redis requires. Table II lists the endpoints.

TABLE II. utility-hub Endpoints

Endpoint	Method	Behaviour
/api/shorten	POST	Validates the URL, creates a 7-character code, returns 201 with the short URL
/:code	GET	Increments the click counter and redirects (302) to the original URL; 404 if unknown
/api/stats/:code	GET	Returns the original URL and its click count
/api/digest	GET	Returns the latest digest written by stats-cron; 404 until one exists
/health	GET	Pings Redis; 200 if reachable, 503 if not
/metrics	GET	Prometheus metrics, including http_requests_total by method, route and status

A request to shorten a URL is handled in four steps. The body is checked for a url field that is a string; the value is parsed with the WHATWG URL parser, and anything that fails to parse is rejected with status 400; a 7-character code is generated with nanoid; and two keys are written, one holding the URL and one holding a click counter set to zero. The response contains the code and a short URL built from the request's protocol and host, so the same code works locally and in production without configuration. Table III shows the key layout.

TABLE III. Redis Key Design

Key	Type	Value
url:<code>	string	Original long URL
url:<code>:clicks	integer	Click counter, incremented with INCR
digest:latest	string (JSON)	repo, stars, forks, openIssues, fetchedAt

The redirect route matches any single path segment, so it explicitly skips the reserved names api, health and metrics to avoid treating them as short codes. Because portfolio is served from a different origin, utility-hub sets a permissive Access-Control-Allow-Origin header; none of its endpoints return private data. Every request is logged as a JSON line by pino-http, and a small middleware increments a Prometheus counter labelled with the method, route and status code once the response has finished [16].

B. *stats-cron*

stats-cron is a script that runs once and exits. fetchStats.js sends an unauthenticated request to the GitHub REST API for the configured repository, maps the response to a digest containing stars, forks, open issues and a timestamp, and throws on any non-2xx status. index.js writes the digest to the digest:latest key and closes the Redis connection in a finally block, so the process ends even when the fetch fails. A failure leaves the process with exit code 1, which marks the workflow run as failed in GitHub Actions and leaves the previous digest in place.

The fetch function is passed in as a parameter with the global fetch as its default, so the unit tests can replace the network call with a stub. The job tracks the public cloud-suite repository. An earlier version pointed at a private repository, and every run failed, because unauthenticated API requests receive 404 for private repositories. We chose to track a public repository rather than add a token, to keep the job free of secrets other than the Redis URL.

C. *portfolio*

portfolio is a single HTML file with inline CSS and JavaScript and no build step. When it loads, it calls /health on utility-hub, and on a separate to-do application the team deployed earlier, and then requests /api/digest. For each service it shows a status indicator, the response time and a link, and it renders the digest with the time it was last fetched.

Free Render services stop when idle, and the first request after that often fails while the service starts again. The first version of the page checked each service once and showed it as down until the visitor refreshed. The current version retries each check up to three times, four seconds apart, and shows "waking up" in between, before it reports a service as unreachable.

VI. DevOps Pipeline

Fig. 2 shows how a change moves through the pipeline. Every push to main is tested against a real Redis instance, and both images are built only after both test jobs pass. Render's automatic deployment is triggered by the same push and runs in parallel with CI, a gap we return to in Section IX.

A. *Containerisation*

Both services use a two-stage Dockerfile based on node:20-alpine. The first stage installs production dependencies only; the second copies them in with the source code, so development dependencies never reach the final image. The containers run as a non-root user. utility-hub's image also declares a HEALTHCHECK that calls /health every 30 seconds. A .dockerignore file keeps node_modules, .git, .env and the tests out of the build context.

For local development, docker-compose.yml starts Redis and utility-hub, and utility-hub waits until Redis passes its own health check. stats-cron is placed behind a manual Compose profile, so docker compose up does not start it; docker compose run --rm stats-cron runs it once, which is how it behaves in production as well.

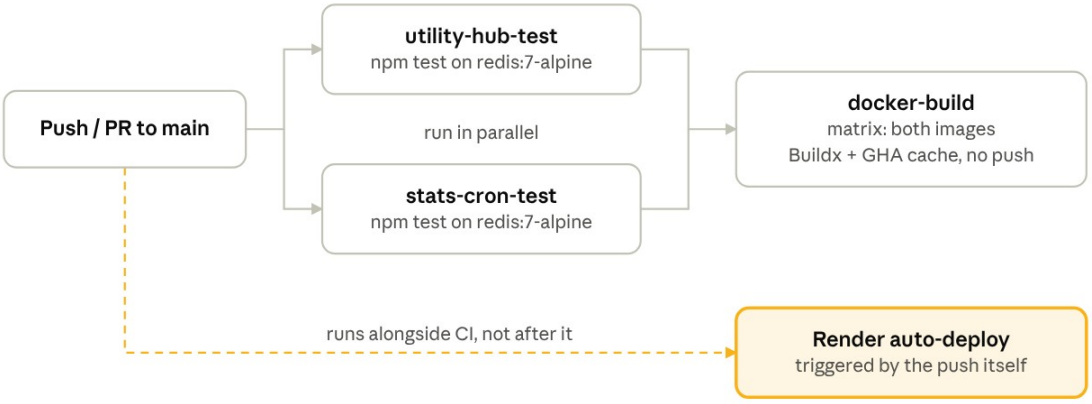


Fig. 2. CI pipeline and deployment path. The Docker build waits for both test jobs; the Render deployment does not wait for CI.

B. *Continuous Integration*

The CI workflow runs on every push and pull request to main. Two jobs, one per service, run in parallel. Each starts a redis:7-alpine service container, waits for it to answer a ping, installs dependencies with npm ci and runs the test suite with REDIS_URL pointing at the container. A third job declares both test jobs as dependencies, so it starts only when both have passed. It uses a matrix to build both images with Docker Buildx and reuses layers through the GitHub Actions cache. Images are built and loaded locally on the runner but are not pushed to a registry.

C. *Scheduled Job*

The stats-cron workflow uses the cron expression 0 */6 * * *, which runs at 00:00, 06:00, 12:00 and 18:00 UTC. It also accepts a manual workflow_dispatch trigger, which we used to run the job on demand during testing and demonstrations. The Redis connection string is read from a repository secret and never appears in the code or the logs, and the connection uses TLS.

D. *Infrastructure as Code*

The Terraform configuration requires Terraform 1.5 or later and version 1.x of the render-oss/render provider [17]. It declares three resources: a free Redis instance with the noeviction policy, a free web service with /health as its health-check path and automatic deployment from main, and a static site for portfolio. The web service's REDIS_URL is taken from the Redis resource's internal connection string, so no credential is written into the configuration, and the Render API key is a sensitive variable supplied through the environment. Terraform state files and variable files are excluded from the repository by .gitignore. There is no cron resource, because the scheduled job runs in GitHub Actions.

The live services were created through Render's dashboard before the configuration was written, so the configuration has not yet been applied. The documented path is to import each existing resource with terraform import and to require terraform plan to report no changes before anything is applied. One attribute path, the Redis internal connection string, is marked in the code as unverified against the provider schema and needs to be checked as part of that import.

E. *Configuration*

Every setting that differs between environments is read from an environment variable, following the Twelve-Factor approach [6]. Table IV lists them.

TABLE IV. ENVIRONMENT VARIABLES

Variable	Used by	Purpose
REDIS_URL	both services	Redis connection string; from Render or a GitHub secret in production
REDIS_TLS	both services	Set to true in production to enable TLS
PORT	utility-hub	Port to listen on; Render sets it, default 3001
DIGEST_REPO	stats-cron	Repository to report on, as owner/name
NODE_ENV	utility-hub	Disables request logging when set to test

VII. FREE-TIER CONSTRAINTS

Running at zero cost was one of our objectives, and the free tiers set limits that shaped the design. Table V summarises them. The limits are those documented by Render and GitHub at the time of writing [13], [14] and may change.

TABLE V. FREE-TIER LIMITS AND THEIR EFFECT

Resource	Limit on the free tier	Effect on the design
Render web service	Stops after 15 minutes without traffic	First request is slow; portfolio retries its checks
Render Key Value	No persistence to disk	Data is lost if the instance restarts
Render cron job	Not available	Scheduled job moved to GitHub Actions
GitHub Actions schedule	Runs can be delayed under load; disabled after 60 days without repository activity	Digest timestamps vary; the workflow must be re-enabled after long idle periods
GitHub REST API	60 unauthenticated requests per hour	Ample for four runs a day; private repositories are not visible

The most visible effect is the cold start. When nobody has used utility-hub for a while, the platform stops it, and the next request has to wait for it to start again, which can take close to a minute. For a demonstration this is acceptable once the page retries; for a real service it would not be. The lack of persistence is a bigger problem than the cold start. Every short URL is held only in memory, so a restart of the Redis instance removes all of them.

VIII. TESTING AND RESULTS

All 12 automated tests pass against a real Redis 7 instance. The suites were run on 20 October 2026 on commit 20ae5e0 of main, using the same commands as CI. Table VI summarises them.

TABLE VI. AUTOMATED TEST RESULTS

Suite	Tests	What is covered	Result
utility-hub (Jest and Supertest)	9	Health and metrics endpoints; rejection of a missing and an invalid URL; create then redirect; 404 for an unknown code; click counting; digest before and after it is written	9 passed
stats-cron, fetchStats (stubbed fetch)	2	Parsing a successful API response; error on a 404	2 passed
stats-cron, main (real Redis)	1	main() writes digest:latest to Redis	1 passed
Total	12		12 passed, 0 failed

The utility-hub tests drive the HTTP layer and Redis together, and the database is flushed before each test so that results do not depend on order. A wrong key name or a missing await therefore fails a test instead of passing against a mock. Only the outbound GitHub request is stubbed, because a test that depends on a third-party API's availability and rate limit would fail for reasons unrelated to our code.

We also checked the live deployment on 20 October 2026. A request to /health on the hosted utility-hub returned a status of ok with Redis connected, which confirms that the web service reaches the managed Redis over TLS. A request to /api/digest returned a digest for arjitmenon9/cloud-suite with a fetchedAt time of 03:47 UTC that day. This shows that the scheduled job had run and that its write was visible to the web service through the shared store, which is the one path in the system that crosses all three platforms.

IX. SECURITY, RELIABILITY AND LIMITATIONS

The system keeps its secrets out of the code and runs its containers with reduced privileges, but it is a demonstration and several of its protections are weaker than they should be.

A. Measures in Place

Connection strings and API keys exist only in Render environment variables, GitHub Actions secrets and Terraform sensitive variables; none are committed. Containers run as a non-root user. The shorten endpoint rejects a missing, non-string or unparseable URL with status 400. Health is checked at three levels: the Docker HEALTHCHECK, Render's health-check path and the browser-side checks in portfolio. Structured logs and the /metrics endpoint make it possible to see request rates and error codes without reading raw output.

B. Known Limitations

Table VII lists the limitations we found while reviewing the code and the deployment, with the evidence for each and a proposed fix.

TABLE VII. KNOWN LIMITATIONS AND REMEDIES

Limitation	Evidence	Impact	Remedy
URL scheme not restricted	javascript:, data: and ftp: URLs are accepted with status 201	The service can store and redirect to non-web schemes	Allow only http: and https: after parsing [18]
TLS certificate not verified	redis.js sets rejectUnauthorized to false	Traffic is encrypted but the Redis server is not authenticated	Verify against the provider's certificate authority
No data durability	Terraform sets persistence_mode to off on a free Redis	A restart loses every short URL and click count	Paid plan with persistence, or a durable store
Deployment not gated on CI	auto_deploy is true and CI is a separate workflow	A commit that fails its tests can still reach production	Render's wait-for-CI option, or a deploy hook called from CI
Production may not run the image	Terraform declares a native Node runtime with npm install and npm start	The non-root user and HEALTHCHECK apply to CI builds only	Deploy the Docker image itself from a registry
Terraform not yet applied	One attribute path is marked unverified in main.tf	The code describes the system but does not manage it	Import the three resources and confirm a plan with no changes
No rate limiting	No limiter on /api/shorten	The endpoint can be used to create codes in bulk	A per-client limit, for example a token bucket in Redis

The first two rows are the cheapest to fix, each a change of a few lines, and should be addressed before the service is used for anything beyond a demonstration.

X. CONCLUSION AND FUTURE WORK

Cloud Computing Suite meets the four objectives it set out with. It uses eight cloud services and tools, demonstrates always-on, scheduled and static compute, tests and builds every change automatically, and runs at no cost. More than any functional requirement, the free-tier constraint shaped the design: it moved scheduled compute from Render to GitHub Actions, made retry logic necessary in the browser, and accounts for the durability and deployment limits in Table VII.

Building the system taught us several things that the course material describes but that we had not experienced directly. The run-then-exit job moved from Render to GitHub Actions without any change to its code. Free services sleep, so clients have to retry rather than report the first failure. Unauthenticated API calls cannot see private resources, which is obvious in hindsight but cost us a series of failed runs. Tests against a real database caught mistakes that a mocked data layer would have hidden. Finally, resources created by hand have to be imported before Terraform can manage them safely, so it is easier to write the configuration first.

Future work follows from Table VII: restricting URL schemes and verifying TLS certificates, gating deployment on CI, pushing the Docker images to a registry and deploying those, applying the Terraform configuration through import, adding rate limiting, and moving to a Redis plan with persistence.

Individual Contributions

Member	Role	Contribution
Arjit Menon	Team lead, back end	Architecture and service split; utility-hub (Express app, URL shortener, Redis key design, /health); Render deployment and managed Redis
Dhruvishi Amin	Infrastructure as Code	Terraform configuration (Redis, web service, static site, variables, outputs); import procedure for existing resources
Arisha Jaiswal	Scheduled compute	stats-cron (GitHub API client, digest format, Redis writer) and its tests; moved the job to the public repository
Manya Vishwanath	DevOps and CI/CD	GitHub Actions CI pipeline (parallel tests on Redis, Docker build matrix with caching); scheduled workflow and secrets; Dockerfiles and Compose setup
Swasti Gupta	Front end, monitoring, documentation	portfolio page with health and digest checks and cold-start retries; Prometheus metrics and structured logging; README and design documentation

References

[1] P. Mell and T. Grance, "The NIST definition of cloud computing," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Special Publication 800-145, Sep. 2011.

[2] J. Lewis and M. Fowler, "Microservices," martinfowler.com, Mar. 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html

[3] A. Menon, D. Amin, A. Jaiswal, M. Vishwanath, and S. Gupta, "cloud-suite," GitHub repository, 2026. [Online]. Available: https://github.com/arjitmenon9/cloud-suite

[4] E. Jonas et al., "Cloud programming simplified: A Berkeley view on serverless computing," Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2019-3, Feb. 2019.

[5] N. Dragoni et al., "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216.

[6] A. Wiggins, "The Twelve-Factor App," 2011. [Online]. Available: https://12factor.net

[7] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, Mar. 2014.

[8] M. Fowler, "Continuous integration," martinfowler.com, May 2006. [Online]. Available: https://martinfowler.com/articles/continuousIntegration.html

[9] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.

[10] M. Shahin, M. Ali Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.

[11] K. Morris, *Infrastructure as Code*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.

[12] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A systematic mapping study of infrastructure as code research," *Inf. Softw. Technol.*, vol. 108, pp. 65–77, Apr. 2019.

[13] Render, "Render documentation." [Online]. Available: https://render.com/docs (accessed Oct. 20, 2026).

[14] GitHub, "GitHub Actions documentation." [Online]. Available: https://docs.github.com/en/actions (accessed Oct. 20, 2026).

[15] Redis Ltd., "Redis documentation." [Online]. Available: https://redis.io/docs/ (accessed Oct. 20, 2026).

[16] Prometheus Authors, "Prometheus documentation." [Online]. Available: https://prometheus.io/docs/ (accessed Oct. 20, 2026).

[17] Render, "render-oss/render Terraform provider." [Online]. Available: https://registry.terraform.io/providers/render-oss/render/latest/docs (accessed Oct. 20, 2026).

[18] OWASP Foundation, "Unvalidated redirects and forwards cheat sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html (accessed Oct. 20, 2026).